RAPPORT TECHNIQUE

Déploiement d'un réseau SDN à trois commutateurs

Émulation de topologie avec Mininet et gestion centralisée des flux par le contrôleur OpenFlow ONOS et son dashboard web

Environnement technique

VMware Workstation · Ubuntu 22.04 LTS · Mininet · Open vSwitch
Docker · ONOS 3.0.0 · OpenFlow 1.3

2026

Table des matières

1. Introduction et objectifs 3

2. Architecture cible 3

3. Environnement et prérequis 4

4. Déploiement pas à pas 5

4.1 Préparation de la machine virtuelle 5

4.2 Installation de Mininet 5

4.3 Installation de Docker 5

4.4 Déploiement d'ONOS 5

4.5 Topologie Mininet 6

4.6 Script de démarrage automatique 6

5. Le dashboard ONOS 6

6. Protocole de tests et validation 7

6.1 Validation préliminaire 7

6.2 Tests définis pour la topologie à trois commutateurs 7

6.3 Ce que ces tests démontrent 8

7. Difficultés rencontrées et solutions 8

8. Limites et perspectives 8

Annexe A. Code source 9

Annexe B. Commandes de référence 10

1. Introduction et objectifs

Le Software-Defined Networking (SDN) sépare le plan de contrôle, qui décide où envoyer chaque paquet, du plan de données, qui se contente de transmettre. Un contrôleur logiciel centralisé pilote alors l'ensemble des commutateurs via un protocole standard, ici OpenFlow 1.3. Le réseau devient programmable, observable depuis un point unique et modifiable sans reconfigurer chaque équipement.

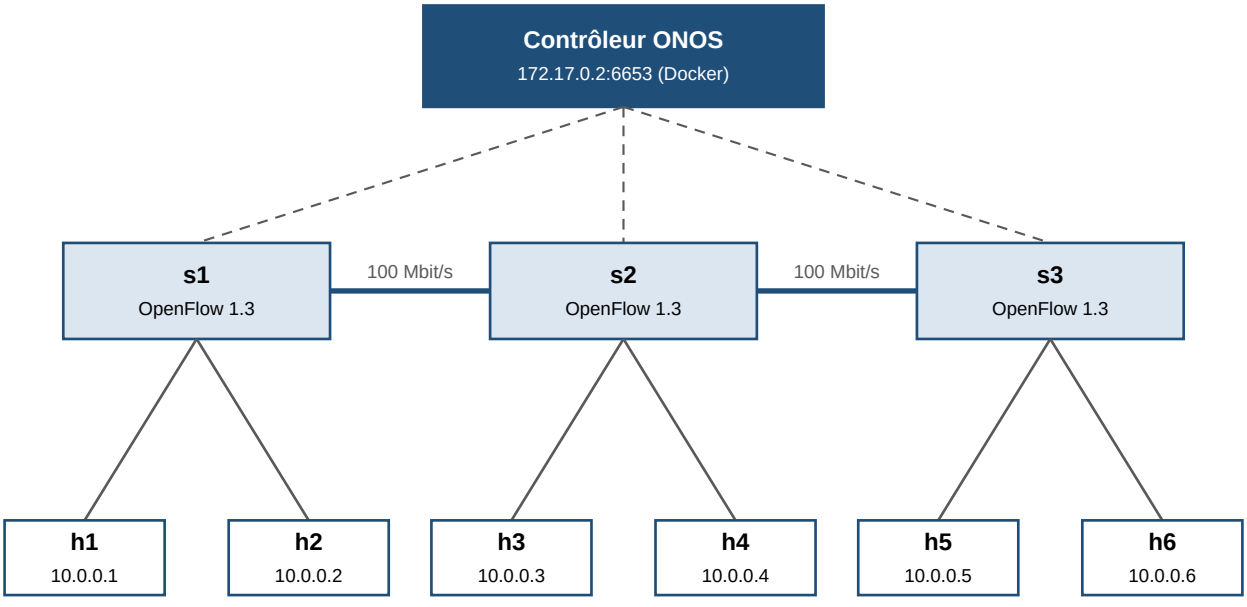
Ce rapport documente le déploiement complet d'un réseau SDN de laboratoire : **trois commutateurs Open vSwitch** et **six hôtes** émulés avec Mininet, pilotés par le contrôleur **ONOS** exécuté dans un conteneur Docker. ONOS a été préféré à Ryu, utilisé lors d'une première version du projet, pour deux raisons : un dashboard web natif (topologie, équipements, hôtes, flux) et une installation par conteneur nettement plus fiable que l'installation Python de Ryu.

Objectifs :

- déployer une VM Linux et y installer Mininet, Docker et ONOS ;
- construire une topologie linéaire à trois commutateurs et la connecter à ONOS ;
- automatiser le démarrage de l'ensemble en une seule commande ;
- exploiter le dashboard ONOS pour visualiser et vérifier le réseau ;
- définir un protocole de tests de connectivité, de performance et de résilience.

2. Architecture cible

La topologie est volontairement **linéaire** (s1 - s2 - s3). Une topologie en triangle créerait une boucle : sans protocole anti-boucle comme STP, le forwarding réactif d'ONOS risquerait de provoquer une tempête de broadcast.



liens hôte-switch : 10 Mbit/s

Élément	Détail
Contrôleur	ONOS 3.0.0 dans un conteneur Docker (`onosproject/onos`), adresse 172.17.0.2, OpenFlow sur le port 6653
Commutateurs	s1, s2, s3 : Open vSwitch, OpenFlow 1.3, mode `fail_mode: secure`
Hôtes	h1 à h6, adresses 10.0.0.1 à 10.0.0.6 (/24), deux hôtes par commutateur
Liens hôte - commutateur	10 Mbit/s (limitation par TCLink)
Liens inter-commutateurs	s1-s2 et s2-s3, 100 Mbit/s

3. Environnement et prérequis

Composant	Choix retenu
Hyperviseur	VMware Workstation Pro (gratuit pour usage personnel)
Système invité	Ubuntu Desktop 22.04 LTS
Ressources de la VM	6 Go de RAM (4 Go au minimum), 4 vCPU, 30 Go de disque, réseau NAT
Émulation réseau	Mininet (paquet Ubuntu) avec Open vSwitch
Conteneurs	Docker CE (dépôt officiel Docker)
Contrôleur	Image `onosproject/onos:latest` (ONOS 3.0.0)
Outils	curl, netcat (nc), git, open-vm-tools

ONOS est une application Java/Karaf gourmande : la VM doit disposer d'au moins 4 Go de RAM, 6 Go étant recommandés. La virtualisation matérielle (Intel VT-x ou AMD-V) doit être activée dans le BIOS.

4. Déploiement pas à pas

4.1 Préparation de la machine virtuelle

Après création de la VM sous VMware et installation d'Ubuntu, le système est mis à jour et les outils de base sont installés :

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y open-vm-tools open-vm-tools-desktop curl netcat-openbsd git
sudo reboot
```

4.2 Installation de Mininet

```
sudo apt install -y mininet
sudo mn --version
sudo mn --test pingall      # doit afficher 0% dropped
sudo mn -c                  # nettoyage
```

4.3 Installation de Docker

```
sudo apt install -y ca-certificates gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo $VERSION_CODENAME) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io
sudo usermod -aG docker $USER
sudo systemctl enable docker
sudo reboot

docker run hello-world      # après redémarrage : vérification
```

4.4 Déploiement d'ONOS

Le conteneur est lancé avec l'option `--restart unless-stopped`, qui le fait redémarrer automatiquement à chaque démarrage de la VM :

```
docker pull onosproject/onos:latest
docker run -t -d --name onos \
  --restart unless-stopped \
  -p 8181:8181 -p 8101:8101 -p 6653:6653 -p 6640:6640 \
  onosproject/onos:latest

docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' onos
```

Port	Usage
8181	Dashboard web et API REST
8101	Console d'administration ONOS (SSH)
6653	OpenFlow : connexion des commutateurs
6640	OVSDB : gestion des Open vSwitch

Le démarrage complet prend de une à trois minutes. Deux points sont essentiels : l'application `org.onosproject.openflow` doit être **activée explicitement** pour qu'ONOS écoute sur le port 6653, et `org.onosproject.fwd` (commutation réactive de niveau 2) doit l'être aussi, faute de quoi les commutateurs, en mode *secure*, ne transmettent aucun trafic. Le script de démarrage (section 4.6) active ces applications automatiquement via l'API REST.

4.5 Topologie Mininet

La topologie est décrite dans `topology_onos.py` (code complet en annexe A). Les commutateurs sont créés avec `protocols='OpenFlow13'` ; le contrôleur est déclaré comme `RemoteController`. L'adresse d'ONOS n'est pas écrite en dur : elle est lue dans la variable d'environnement `ONOS_IP`, fournie par le script de démarrage, ce qui évite toute modification si l'adresse du conteneur change.

4.6 Script de démarrage automatique

Un réseau Mininet vit uniquement en mémoire (interfaces virtuelles et espaces de noms Linux) : il disparaît à chaque arrêt de la VM et ne peut pas être rendu persistant. Le script `start-sdn.sh` (annexe A) le reconstruit en une commande, après chaque redémarrage :

```
~/sdn-lab/start-sdn.sh
```

Il enchaîne les étapes suivantes :

- démarrage du conteneur ONOS s'il est arrêté, puis lecture de son adresse IP ;
- attente que l'API REST d'ONOS réponde (et non une lecture des journaux, qui conservent les lignes des exécutions précédentes et donneraient un faux signal) ;
- activation des applications openflow, lldpprovider, hostprovider et fwd par l'API REST ;
- attente de l'ouverture du port TCP 6653, avec abandon explicite au bout de trois minutes ;
- nettoyage de Mininet puis lancement de la topologie.

5. Le dashboard ONOS

Une fois le prompt `mininet>` affiché, le dashboard est accessible dans le navigateur de la VM à l'adresse `http://localhost:8181/onos/ui` (identifiants par défaut : **onos** / **rocks**).

Vue (menu)	Contenu
Topology	Carte interactive. Touche **H** : afficher les hôtes ; **L** : changer les étiquettes ; clic sur un nœud : détails ; Shift + clic sur deux hôtes : créer un intent hôte à hôte. Le panneau « ONOS Summary » résume équipements, liens, hôtes et flux.
Devices	Les trois commutateurs, leur état, leurs ports et leur version OpenFlow
Hosts	Les six hôtes avec adresses MAC et IP, et le port de rattachement
Links	Les liens découverts entre commutateurs (comptés dans les deux sens)
Flows	Les règles installées sur chaque commutateur, avec compteurs de paquets : équivalent graphique de `ovs-ofctl dump-flows`
Applications	Applications ONOS actives ; vérifier openflow et fwd

Le dashboard n'a pas de fonction « ping » : ONOS est un contrôleur et non un hôte du réseau. Les pings se lancent depuis Mininet ; leur effet s'observe dans le dashboard (compteurs de la vue Flows, affichage du trafic sur la carte). Les hôtes n'apparaissent qu'après avoir émis du trafic : un `pingall` préalable les fait découvrir.

6. Protocole de tests et validation

6.1 Validation préliminaire

Le même montage, réalisé d'abord avec deux commutateurs et quatre hôtes, a été validé : le panneau Summary indiquait 2 équipements, 2 liens (un lien physique compté dans les deux sens), 4 hôtes et 8 flux, et `pingall` affichait 0 % de perte (12/12). La configuration à trois commutateurs reprend à l'identique les mêmes composants.

6.2 Tests définis pour la topologie à trois commutateurs

Le tableau suivant décrit chaque test, la commande à exécuter dans le CLI Mininet (ou le terminal) et le comportement attendu.

Test	Commande	Résultat attendu
Connectivité globale	`pingall`	0 % dropped (30/30 received)
Même commutateur	`h1 ping -c 4 h2`	0 % de perte, latence faible
Traversée de 3 commutateurs	`h1 ping -c 4 h6`	0 % de perte, latence un peu supérieure
Débit	`iperf h1 h2` puis `iperf h1 h6`	Environ 9 à 10 Mbit/s (liens hôtes limités à 10)
Découverte ONOS	Dashboard, panneau Summary	Devices 3, Links 4, Hosts 6
Flux installés	`sudo ovs-ofctl -O OpenFlow13 dump-flows s2` et vue Flows	Règles cohérentes avec le dashboard, compteurs croissants
Coupure s2-s3	`link s2 s3 down` puis `h1 ping -c 3 h5`	100 % de perte ; lien absent du dashboard, Links passe à 2
Isolation partielle	`h1 ping -c 3 h3` (pendant la coupure)	0 % de perte : s1-s2 est intact
Rétablissement	`link s2 s3 up` puis `h1 ping -c 3 h5`	0 % de perte, éventuellement après quelques secondes ; Links revient à 4
Coupure s1-s2	`link s1 s2 down` puis `h1 ping -c 3 h3`	100 % de perte ; rétablir avec `link s1 s2 up`
Panne d'un hôte	`h6 ifconfig h6-eth0 down` puis `h1 ping -c 3 h6`	Échec du ping ; h6 peut rester affiché (monitorHosts désactivé dans ONOS)

Test	Commande	Résultat attendu
Redémarrage du contrôleur	`docker restart onos`	Pertes pendant environ 2 minutes puis reconnexion ; sinon `sudo mn -c` et relancer le script

Les valeurs de la colonne « résultat attendu » décrivent le comportement normal de la topologie ; les mesures effectives (sorties de `pingall`, `iperf`, captures du dashboard) peuvent être jointes au dépôt à côté de ce rapport.

6.3 Ce que ces tests démontrent

La coupure d'un lien illustre la propriété centrale du SDN : le contrôleur met à jour sa vision du réseau (topologie, chemins) de façon visible et immédiate dans le dashboard, sans reconfiguration manuelle des commutateurs. La topologie linéaire n'offrant aucun chemin de secours, une coupure isole simplement un segment ; une étude de la redondance nécessiterait une topologie maillée et un mécanisme anti-boucle.

7. Difficultés rencontrées et solutions

Symptôme	Cause	Solution
Port 6653 fermé (« Connection refused »)	ONOS pas encore démarré, ou application *openflow* non activée	Attendre le démarrage complet ; activer org.onosproject.openflow (fait par le script)
Devices 0 dans le dashboard	Commutateurs connectés avant qu'ONOS soit prêt	`sudo mn -c` puis relancer start-sdn.sh
pingall à 100 % de perte alors qu'ONOS est connecté	Application *fwd* inactive ; en mode *secure*, un commutateur sans règle bloque tout	Activer org.onosproject.fwd (fait par le script)
Fichier Python introuvable ou tronqué	Sauvegarde nano incomplète, ou collage altéré dans le terminal de la VM	Copier les fichiers téléchargés ; vérifier avec `python3 -m py_compile` et `wc -l` (un simple `grep` ne détecte pas un fichier tronqué)
ONOS arrêté après redémarrage de la VM	Conteneur sans politique de redémarrage	`docker run --restart unless-stopped` ou `docker update --restart unless-stopped onos`
Topologie disparue après redémarrage	Mininet vit en mémoire : comportement normal	Relancer start-sdn.sh
Hôtes « fantômes » après changement de topologie	ONOS conserve les anciens équipements	`docker restart onos`
Attente trop courte ou faussée dans le script	`docker logs` conserve les lignes des exécutions précédentes	Tester l'API REST et le port TCP plutôt que les journaux

8. Limites et perspectives

- Contrôleur unique : en production, ONOS est déployé en grappe pour éviter un point de défaillance.
- Topologie sans redondance : un lien s1-s3 supplémentaire exigerait un mécanisme anti-boucle ou une application de routage par chemins.
- Identifiants par défaut (onos / rocks) : acceptables en laboratoire uniquement.
- Pistes : temps de convergence après rétablissement d'un lien, QoS, collecte de statistiques via l'API REST, application ONOS personnalisée.

Annexe A. Code source

topology_onos.py

```
#!/usr/bin/env python3
import os
from mininet.net import Mininet
from mininet.node import RemoteController, OVSSwitch
from mininet.cli import CLI
from mininet.link import TCLink
from mininet.log import setLogLevel, info

# L'IP d'ONOS est fournie par start-sdn.sh (variable d'environnement)
ONOS_IP = os.environ.get('ONOS_IP', '172.17.0.2')


def topology():
    net = Mininet(controller=RemoteController, switch=OVSSwitch,
                  link=TCLink, autoSetMacs=True)

    info('*** Ajout du controleur ONOS (%s)\n' % ONOS_IP)
    c0 = net.addController('c0', controller=RemoteController,
                           ip=ONOS_IP, port=6653)

    info('*** Ajout des 3 switches OpenFlow 1.3\n')
    s1 = net.addSwitch('s1', protocols='OpenFlow13')
    s2 = net.addSwitch('s2', protocols='OpenFlow13')
    s3 = net.addSwitch('s3', protocols='OpenFlow13')

    info('*** Ajout des hotes\n')
    h1 = net.addHost('h1', ip='10.0.0.1/24')
    h2 = net.addHost('h2', ip='10.0.0.2/24')
    h3 = net.addHost('h3', ip='10.0.0.3/24')
    h4 = net.addHost('h4', ip='10.0.0.4/24')
    h5 = net.addHost('h5', ip='10.0.0.5/24')
    h6 = net.addHost('h6', ip='10.0.0.6/24')

    info('*** Liens hotes - switches\n')
    net.addLink(h1, s1, bw=10)
    net.addLink(h2, s1, bw=10)
    net.addLink(h3, s2, bw=10)
    net.addLink(h4, s2, bw=10)
    net.addLink(h5, s3, bw=10)
    net.addLink(h6, s3, bw=10)

    info('*** Liens inter-switch (topologie lineaire, sans boucle)\n')
    net.addLink(s1, s2, bw=100)
    net.addLink(s2, s3, bw=100)

    info('*** Demarrage du reseau\n')
    net.build()
    c0.start()
    s1.start([c0])
    s2.start([c0])
    s3.start([c0])

    info('*** Reseau demarre. Lancement du CLI Mininet.\n')
    CLI(net)

    info('*** Arret du reseau\n')
    net.stop()


if __name__ == '__main__':
    setLogLevel('info')
    topology()
```

start-sdn.sh

```
#!/bin/bash
AUTH="onos:rocks"

echo "*** Demarrage du conteneur ONOS (si arrete)..."
docker start onos > /dev/null 2>&1

ONOS_IP=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' onos 2>/dev/null)
if [ -z "$ONOS_IP" ]; then
    echo "ERREUR : conteneur 'onos' introuvable ou arrete. Verifiez avec : docker ps -a"
    exit 1
fi
echo "*** IP d'ONOS : $ONOS_IP"

echo "*** Attente de l'API d'ONOS (1 a 3 minutes)..."
until curl -sf -u "$AUTH" "http://$ONOS_IP:8181/onos/v1/applications" > /dev/null; do
    echo "    ONOS demarre encore..."
    sleep 5
done

echo "*** Activation des applications ONOS (openflow, decouverte, fwd)..."
for app in openflow lldpprovider hostprovider fwd; do
    curl -s -u "$AUTH" -X POST \
      "http://$ONOS_IP:8181/onos/v1/applications/org.onosproject.$app/active" > /dev/null
done

echo "*** Attente du port OpenFlow 6653..."
for i in $(seq 1 60); do
    nc -z "$ONOS_IP" 6653 && break
    sleep 3
done
if ! nc -z "$ONOS_IP" 6653; then
    echo "ERREUR : le port 6653 reste ferme apres 3 minutes."
    echo "Regardez les logs : docker logs --tail 50 onos"
    exit 1
fi

echo "*** Nettoyage Mininet..."
sudo mn -c > /dev/null 2>&1

echo "*** Lancement de la topologie..."
sudo ONOS_IP="$ONOS_IP" python3 ~/sdn-lab/topology_onos.py
```

Annexe B. Commandes de référence

Commande	Usage
`~/sdn-lab/start-sdn.sh`	Démarrer (ou redémarrer) l'ensemble du réseau
`sudo mn -c`	Nettoyer les résidus Mininet
`docker ps -a`	État du conteneur ONOS
`docker logs --tail 50 onos`	Dernières lignes du journal d'ONOS
`docker restart onos`	Redémarrer ONOS (purge des équipements fantômes)
`nc -zv 172.17.0.2 6653`	Tester l'ouverture du port OpenFlow
`sudo ovs-vsctl show`	Configuration des commutateurs Open vSwitch
`sudo ovs-ofctl -O OpenFlow13 dump-flows s1`	Flux installés sur s1
`mininet> link s2 s3 down | up`	Couper / rétablir un lien
`ssh -p 8101 onos@localhost`	Console ONOS (mot de passe : rocks)